

İSTEMCİ VE SUNUCU TEMELLERİ

[Kaynak](#)

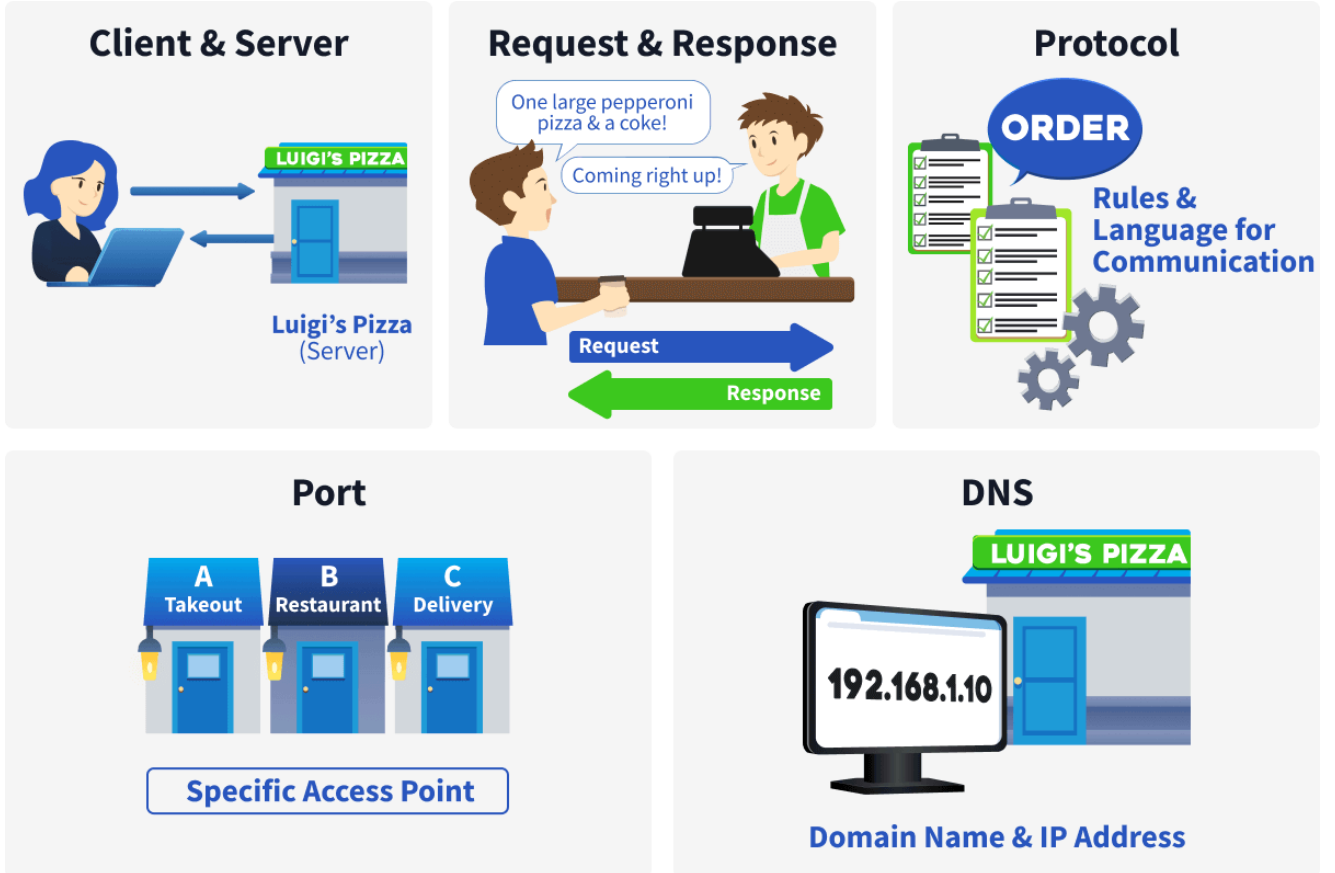
İÇERİK

1. Client-Server Modeli: Pizza Siparişi Analojisiyle Ağ Temelleri
2. Uygulamada Web İletişimi

Client-Server Modeli: Pizza Siparişi Analojisiyle Ağ Temelleri

Bilgisayar sistemlerinin birbirlerine nasıl hizmet (service) sunduğunu anlamamanın en kolay yolu, bunu günlük hayattaki işletme-müşteri ilişkilerine benzetmektir. Bu notlarda, karmaşık ağ yapılarını ve istemci-sunucu modelini bir "pizza siparişi" analojisi üzerinden kafamıza tamamen oturtacağız.

Bu süreç aslında her gün yaşadığımız ama içselleştirdiğimiz için üzerine düşünmediğimiz bir mekanizma. Ağ sistemleri de tam olarak bu mantıkla çalışıyor.



İstemci, Sunucu ve Hizmet (Service, Client, Server)

Cuma akşamı Alice ve Bob pizza sipariş etmek istiyor. Alice menüye bakıp ne istediğini Bob'a söylüyor, Bob arabaya atlayıp Luigi'nin Pizzacısı'na (Luigi's Pizza) gidiyor ve siparişi

veriyor. Sipariş hazırlandığında Bob pizzayı alıp eve dönüyor.

Şimdi bu mantığı bilgisayar sistemlerine oturtalım:

- **İstemci (Client):** Bizim senaryomuzda Alice'tir. İsteği oluşturan taraftır. Bilgisayar dünyasında bu, bir web sitesine gitmek için kullandığımız **web tarayıcısıdır** (Chrome, Firefox vb.).
- **Sunucu (Server):** Luigi'nin Pizzacısı'dır. İsteği karşılayan ve hizmeti sağlayan sistemdir. Bize web sayfasını barındıran ve gönderen makinedir.
- **Hizmet (Service):** Pizzacının sunduğu "paket servis" imkanıdır.

Buradaki en kritik kural şudur: İletişimi ve isteği **her zaman istemci (client) başlatır**. Sunucu durup dururken size veri göndermez; önce istemcinin gidip o veriyi (örneğin web sayfasını) talep etmesi gerekir.

İstek ve Yanıt (Request and Response)

Alice, Bob aracılığıyla "Büyük boy pepperoni pizza ve kola" **isteğinde (request)** bulunur. Dikkat edilmesi gereken nokta şudur: Alice bu isteği Bob'dan yapmıyor, Bob sadece isteği taşıyan bir aracı (ağ/network). İstek doğrudan sunucuya (Luigi'ye) gidiyor.

Bu süreçte işlerin ters gitme ihtimali de vardır. Eğer istek yanlış bir formatta iletilirse veya istenen kaynak mevcut değilse, bir **hata yanıtı (error response)** alırız. Örneğin Bob eve dönüp "Pepperoni kalmamış" veya "Pizzacı siparişi hiç anlamadı" diyebilir.

Sistemlerde de bu birebir aynıdır: İstemci (tarayıcı) sunucudan bir web sayfası talep eder. Eğer sayfa bulunamazsa veya istek bozuksa, sunucu bize o sayfayı göndermek yerine bir hata kodu döndürür (Örn: sayfa yoksa 404 Not Found dönmesi gibi). Her şey yolundaysa web sayfası bir **yanıt (response)** olarak istemciye iletilir.

Protokol (Protocol)

Yurtdışına çıkana kadar konuştuğumuz dilin ne kadar önemli bir iletişim kuralı olduğunu fark etmeyiz. Alice siparişini oluştururken, Luigi'nin Pizzacısı'nın anlayacağı bir **dil** kullanır. Sadece bu da değil, ne sipariş edebileceğini görmek için pizzacının **menüsündeki** kurallara uyar. Bob bu isteği (dili ve siparişi) anlar, pizzacıya iletir, pizzacının verdiği yanıtı anlar ve geri getirir.

Bilgisayar ağlarında Bob'un (aracının) üstlendiği bu role ve konuşulan ortak dile **Protokol (Protocol)** diyoruz. Protokol, bir istemcinin bir sunucuyla tam olarak **nasıl iletişim kuracağını belirleyen katı kurallar bütünüdür**. Eğer bu kurallar olmazsa iletişim çöker. Bir protokolün tanımladığı temel unsurlar şunlardır:

- **Komutlar:** İstemci ve sunucunun hangi eylemleri anlayabildiği. (Örnek: *Web sunucularında veri çekmek için kullanılan GET komutu.*)
- **Yapı (Structure):** İsteğin hangi sırayla ve nasıl dizileceği. (Örnek: *Önce komutun belirtilmesi, ardından istenen siparişin/dosyanın yazılması.*)

- **Sözdizimi (Syntax):** Hangi dilin veya formatın kullanıldığı. (Örnek: *Alice'in İngilizce konuşması veya sistemlerin HTTP formatını kullanması.*)
- **Başarılı Yanıtlar:** Belirli bir isteğe karşılık ne tür bir yanıt verilmesi gerektiği. (Örnek: *Pizza siparişine karşılık pizzanın verilmesi = GET isteğine karşılık HTML sayfasının dönmesi.*)
- **Hatalı Yanıtlar:** Geçersiz, bozuk veya ulaşılamaz isteklere ne cevap verileceği. (Örnek: *Sunucunun "Pepperoni pizza kalmadı" demesi.*)

Port (Bağlantı Noktası)

Bir sistem üzerinde çalışan belirli bir **hizmeti (service)** tanımlamak ve ona ulaşmak için **Port** kullanılır. Bir istemci, sunucudaki bir hizmete erişmek istediğinde mutlaka doğru porta bağlanmak zorundadır.

Şöyle düşünün: Luigi'nin Pizzacısı'na paket servis almak için giden herkesin girmek zorunda olduğu **özel bir kapı** var. Herkes aynı kapıyı kullanır. İşte bir sunucudaki hizmet de tam olarak bu şekilde belirli bir portu "dinler" (listening).

Diyelim ki Luigi işleri büyüttü ve aynı binada birden fazla hizmet veriyor: Paket servis, restoranda yeme (dining in) ve evlere teslimat. İçeride karmaşa çıkmaması için her hizmete ayrı bir kapı tahsis edilir:

- A Kapısı -> Paket Servis
- B Kapısı -> Restoran
- C Kapısı -> Evlere Teslimat

İşte tek bir sunucu makinesi de aynı anda birden fazla hizmeti (Örn: hem Web sunucusu hem Veritabanı) çalıştırabilir. Dışarıdan gelen isteğin hangi hizmete gitmesi gerektiğini **Port numaraları** belirler. Yanlış kapıyı (portu) çalarsanız, istediğiniz hizmete ulaşamazsınız.

DNS (Domain Name Service - Alan Adı Sistemi)

Alice, Bob'a siparişi verdiğinde bilinen tek şey mekanın ismiydi: "Luigi's Pizza". Sadece bu isimle yola çıkıp hedefe varmak imkansızdır. Bu yüzden Bob, mekanın ismini GPS cihazına yazar ve cihaz ona Luigi'nin **fiziksel koordinatlarını** verir. Bob bu koordinatları takip ederek hedefe ulaşır.

Bilgisayar ağlarında bu GPS sisteminin birebir karşılığı **DNS (Domain Name Service)**'tir. Siz tarayıcınıza bir web sitesinin ismini girdiğinizde, DNS bu insan tarafından okunabilen ismi alır ve o sunucunun asıl konumuna, yani **IP Adresine (Internet Protocol Address)** çevirir.

Bilgisayarlar isimlerle değil, bu IP adresleriyle iletişim kurar. IP adresi, bilgisayar sistemleri için tıpkı ev adresiniz (sokak adı, bina numarası, posta kodu, şehir ve ülke) gibidir. Eğer DNS olmasaydı, internette girmek istediğimiz her web sitesinin sadece rakamlardan oluşan karmaşık IP adreslerini ezberlemek zorunda kalırdık. İstek gönderdiğimizde arka planda

DNS bizim yerimize "Bu isim hangi adrese denk geliyor?" sorusunu çözer (resolve) ve bizi doğru hedefe yönlendirir.

Uygulamada Web İletişimi

Web uygulamalarının birbiriyle nasıl konuştuğunu anlamak, siber güvenliğin temelidir. Sistemlerin arka planda bizim adımıza neler yaptığını ve bu iletişimin kurallarını net bir şekilde oturtmamız gerekiyor. İşte bu notlarda, teoriden çıkıp işin mutfağına, tarayıcımızın arka planda çevirdiği işlere bakıyoruz.

HTTP(S) ve Durumsuzluk (Statelessness) İlkesi

Web dünyasının temel iletişim protokolü olan **HTTP(S)** (Hypertext Transfer Protocol Secure), yapısı gereği "**stateless**" (durumsuz) bir protokoldür.

Peki "stateless" olmak ne demek? İşin mantığı şu: Sunucu (server) tamamen balık hafızalıdır. Yapılan her bir istek (request) birbirinden tamamen bağımsız olarak işlenir. Sunucu, senin bir saniye önce yaptığın istekle ilgili hiçbir bilgiyi aklında tutmaz.

Neden böyle ve biz nasıl "Login" kalabiliyoruz? Protokolün kendisi durumsuz olsa da, modern web siteleri ve web uygulamaları bu sorunu çözmek için **Uygulama Seviyesinde (Application Level)** bazı mekanizmalar kullanırlar.

Örneğin, bir siteye kullanıcı adı ve şifrenle giriş yaptığında:

1. Sunucu sana özel bir **session identifier** (oturum tanımlayıcı) oluşturur.
2. Bu tanımlayıcı genellikle bir **cookie** (çerez) veya **token** (jeton) içine yerleştirilir.
3. Sen site içinde başka bir sayfaya tıkladığında (yani yeni bir istek yaptığında), tarayıcın bu cookie/token'ı da istekle beraber sunucuya gönderir.

Bunu yapmazsak ne olur? Eğer bu oturum mekanizmaları olmasaydı, sunucu kim olduğunu asla hatırlamayacağı için sitede tıkladığın *her bir linkte* tekrar kullanıcı adı ve şifre girmek zorunda kalırdın.

HTTP Komutları (Metotları)

HTTP'nin kurallarını tanımlayan ana spesifikasyonlarda (bunlara resmi dilde **RFC - Request for Comments** dökümanları denir), iletişimi sağlamak için tanımlanmış **9 temel komut** bulunur. Web jargonunda biz bunlara "komut" değil, **Method (Metot)** diyoruz.

Karşımıza çıkacak temel metotların tam listesi şudur:

- **GET**
- **POST**
- **PUT**
- **DELETE**
- **PATCH**

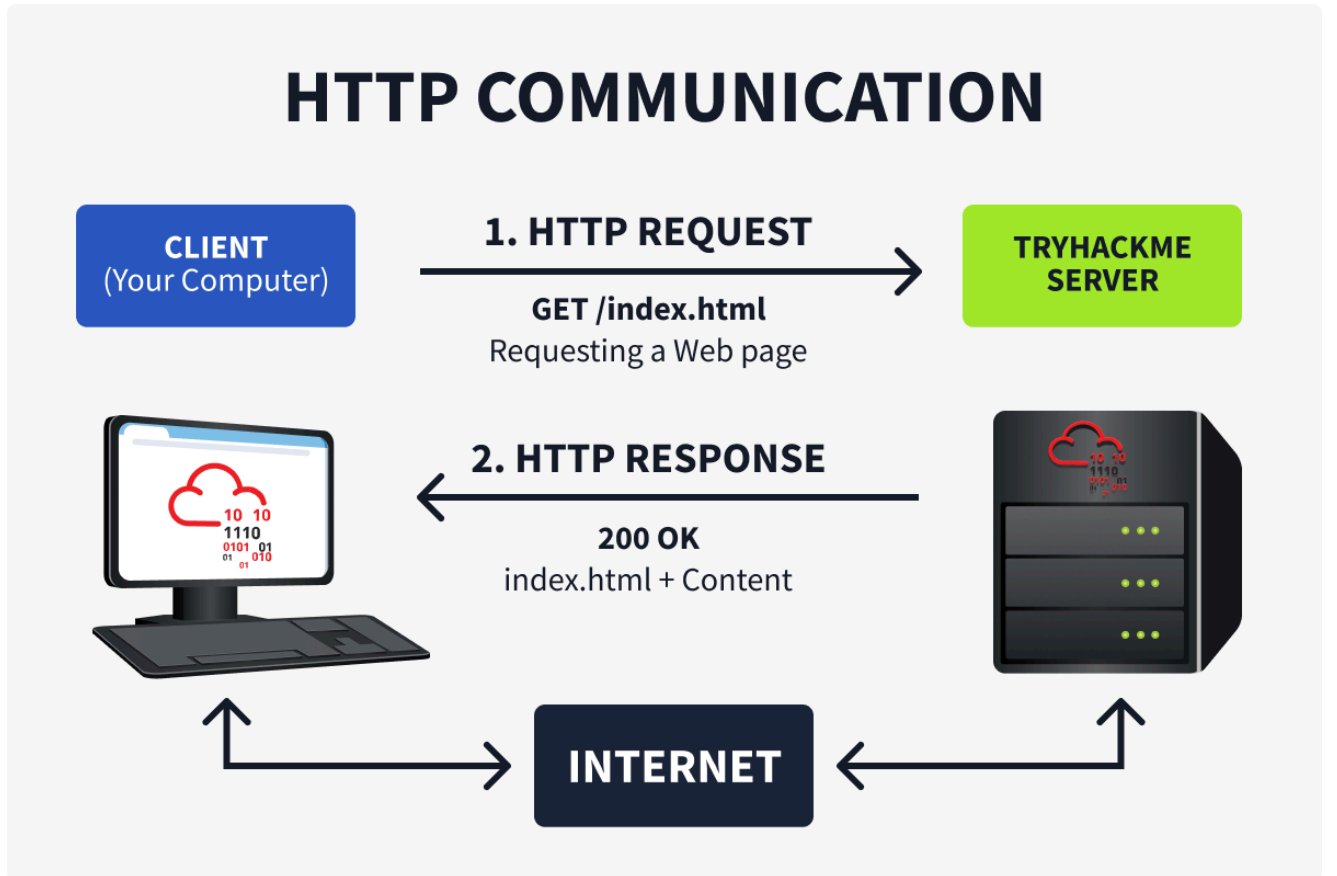
- HEAD
- OPTIONS
- CONNECT
- TRACE

Bu metotların her biri sunucuya farklı bir niyetle geldiğimizi belirtir. Biz şimdi en yaygın olan ve her gün farkında olmadan binlerce kez kullandığımız **GET** metoduna odaklanacağız ve bunu pratik bir şekilde tarayıcı üzerinden inceleyeceğiz.

GET Metodu ve Pratik Ağ Analizi

GET metodu, adından da anlaşılacağı gibi oldukça basittir: Amacı web sunucusundan bir **kaynak çekmektir (retrieve a resource)**.

Örneğin, tarayıcıya `https://tryhackme.com` yazıp Enter'a bastığında, arka planda sen fark etmeden şöyle bir istek oluşturulur:



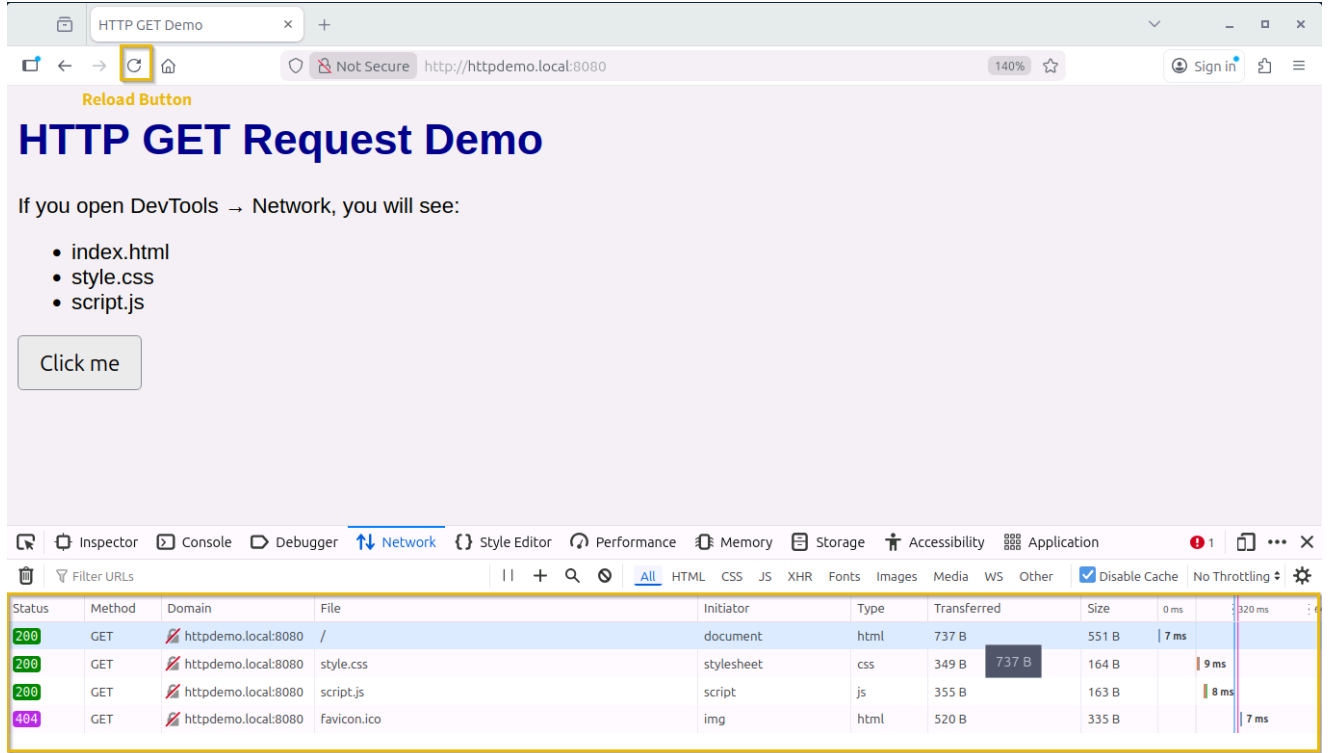
HTTP

```
GET https://tryhackme.com/index.php
```

Senin bu komutu elle yazmana gerek yoktur. İstemci (client - yani tarayıcın), senin girdiğin URL'i ve HTTP kurallarını kullanarak bu mesajı arka planda inşa eder. Sunucu bu isteği aldığı anda, sana bir durum kodu (status code - isteğin akıbetini belirten kod) ve istediğin web sayfasının içeriğini yanıt (response) olarak gönderir.

Tarayıcıda GET İsteğini İncelemek (Walkthrough)

Bu süreci kendi gözlerimizle görmek için **Firefox Geliştirici Araçları'nı (Developer Tools)** kullanacağız. Web trafiğini analiz etmek ve hata ayıklamak (debug) için en sık başvuracağımız yer burasıdır.

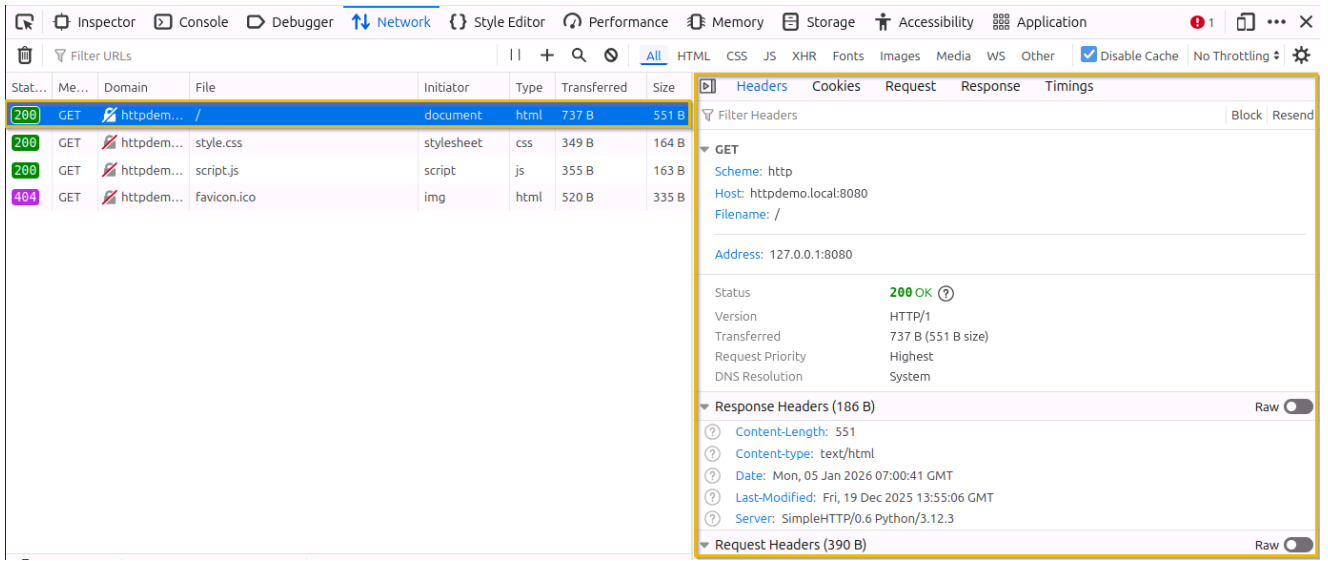


(Not: Laboratuvar ortamında `httpdemo.local:8080` gibi bir test sunucusunda olduğumuzu varsayarak ilerliyoruz.)

1. Tarayıcıda hedef web sayfasına git.
2. Klavyeden **F12**'ye bas veya sayfada sağ tıklayıp **"İncele" (Inspect)** seçeneğini seç. Bu, Geliştirici Araçları panelini açar.
3. Açılan panelde üst sekmelerden **"Network" (Ağ)** sekmesine tıkla. (Burası tarayıcıdan çıkan ve tarayıcıya giren tüm trafiği listeler).
4. Trafiği yakalayabilmek için sayfayı **yenile (Reload)**.

Sayfayı yenilediğinde, Network sekmesinde alt alta birçok GET isteğinin düştüğünü göreceksin. Çünkü bir web sayfası sadece metinden ibaret değildir; resimler, CSS dosyaları, JavaScript dosyaları için arka arkaya birçok GET isteği atılır.

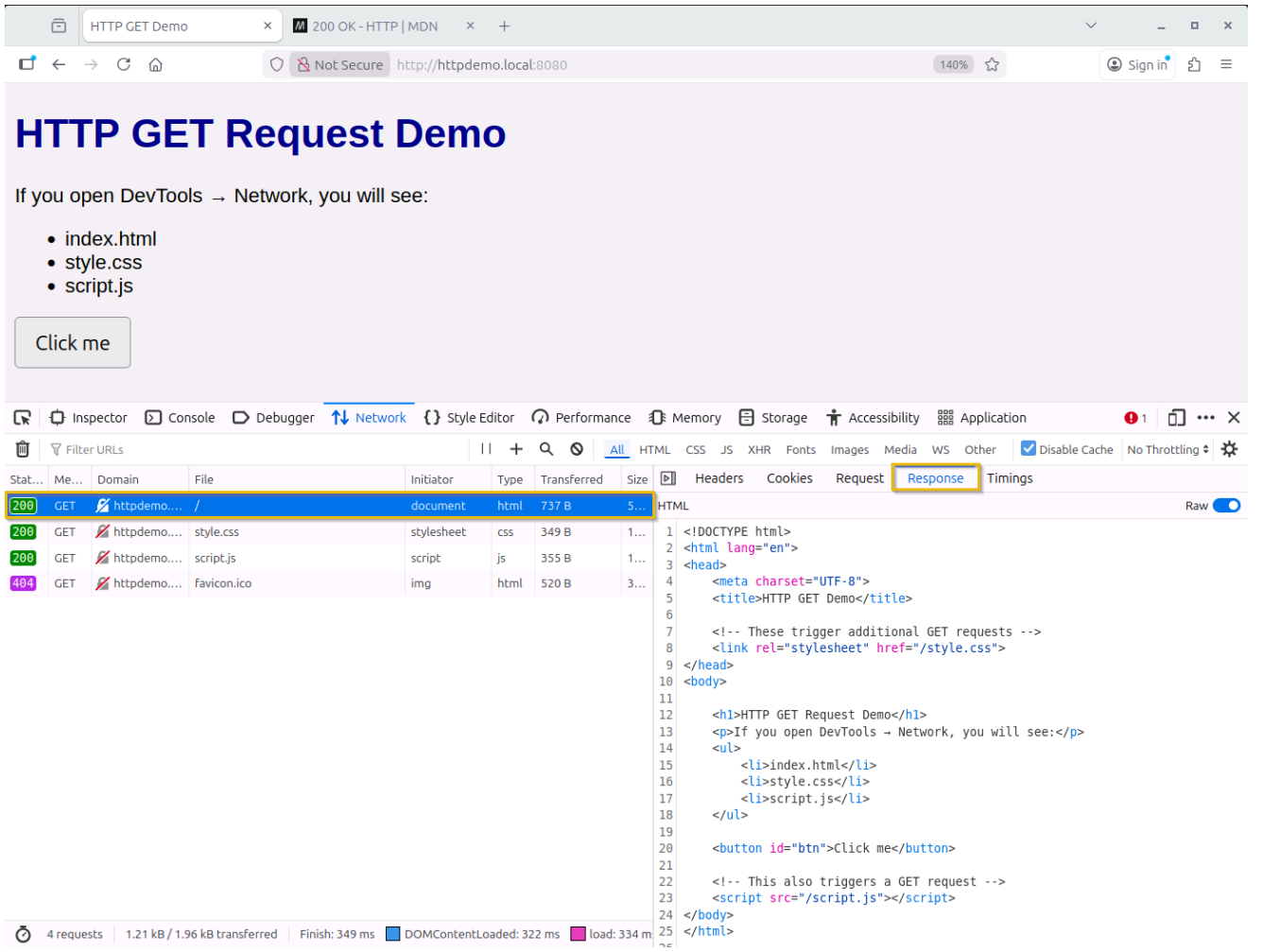
Listedeki ilk isteğe (genellikle ana HTML dosyasının isteğidir) tıkladığımızda, sağ tarafta bir detay paneli açılır. Burada yakaladığımız paketin başlıklarını (headers) görürüz.



Yakalanan Çıktının (Parametrelerin) Yorumlanması

Sağ panelde karşımıza çıkan kritik alanların anlamları şunlardır:

- **Scheme (Şema):** Hangi protokolün kullanıldığını söyler. Çıktıda HTTP veya güvenli versiyonu olan HTTPS yazar.
- **Host:** Kaynak talep ettiğimiz sunucunun adıdır (Örn: httpdemo.local).
- **Filename (Dosya Adı):** Sunucudan tam olarak hangi dosyayı istediğimizi gösterir. Çıktıda genellikle / (kök dizin) işareti görürüz. Web sunucusu mantığında / dizinine gitmek, arka planda varsayılan olarak index.html (veya index.php) dosyasını çağırmak anlamına gelir.
- **Address (Adres):** Web sitesinin barındırıldığı fiziksel IP adresini gösterir. Eğer laboratuvar ortamında sunucu ve istemci aynı makinedeyse (localhost), bu adres 127.0.0.1 olarak görünür.
- **Status (Durum):** İsteğimizin başarılı olup olmadığını sunucunun bize bildirdiği koddur. Çıktıda 200 OK görüyorsak, "İstek başarılı, istediğin dosyayı buldum ve sana gönderiyorum" demektir. (Eğer dosya olmasaydı meşhur 404 Not Found hatasını görecektik).



Response (Yanıt) Yapısı

Biz bir istek gönderdikten sonra sunucudan gelen paket (Response) teknik olarak iki ana parçaya ayrılır:

1. **Response Header (Yanıt Başlığı):** Yanıt hakkındaki üst verileri (metadata) barındırır. Sunucu tipi, tarih, içeriğin boyutu ve yukarıda bahsettiğimiz "Set-Cookie" (oturumu açık tutma) işlemleri bu başlıkların içinde gizlidir.
2. **Response Body (Yanıt Gövdesi):** Bizim asıl talep ettiğimiz içeriktir.

Geliştirici araçlarında bu asıl içeriği görmek için, incelediğimiz isteğin detay panelindeki **"Response" (Yanıt)** sekmesine tıklarız. Bu sekmeye tıkladığımızda, sunucunun bize gönderdiği ham **HTML** kodlarını (index sayfasının kaynak kodunu) tüm çıplaklığıyla görebiliriz.